

Ejercicio 02. Rutas con A* en México

1. Implementación

Adapté A* del proyecto de Rumania para trabajar con el grafo de México. Usé el id de cada ciudad como estado, viajar a un vecino como acción y el campo km de cada arista como costo. El grafo es no dirigido, así que agregué cada conexión en ambos sentidos.

La frontera se ordena por $f=g+h$. Guardo el mejor g conocido y el padre de cada estado para reconstruir la ruta. La meta se acepta cuando sale de la frontera; las entradas antiguas se descartan. El CLI está en `find_route.py` y el mapa calcula la misma búsqueda con `route.js`.

2. Nombres repetidos y heurística

Los nombres no sirven como identificador único. Por ejemplo, Puebla aparece con id 4 en Puebla y con id 580 en Baja California. En la terminal, un nombre ambiguo produce un aviso con las opciones; se debe indicar el id o el estado. En el mapa, cada opción muestra nombre, estado e id.

Usé haversine hasta el destino, con radio 6371 km. La distancia directa es una cota inferior por la desigualdad triangular. Como el JSON redondea los pesos a dos decimales, apliqué el factor 0.998517792300 para conservar esa propiedad sin cambiar ninguna arista: $h=\alpha*\text{haversine}$.

El factor es el mínimo entre 1 y $\text{peso}(u,v)/\text{haversine}(u,v)$, con un margen numérico. Entonces $\alpha*d(u,v)\leq\text{peso}(u,v)$ y, por desigualdad triangular, $h(u)\leq\text{peso}(u,v)+h(v)$. Así h es consistente y admisible. Sigue siendo A* con una estimación geográfica, no UCS.

3. Resultados

Ruta	Costo	Saltos	Expand. A*	Expand. UCS
Tijuana -> Cancún	4528.20 km	124	949	998
Mexico City -> Monterrey	1041.87 km	27	428	783

En la ruta larga obtuve 124 saltos y 4528.20 km. Los costos coinciden con UCS, y el navegador devuelve los mismos caminos y costos que Python. El grafo conserva sus 1,000 nodos y 2,565 aristas.

El mapa representa conexiones por proximidad; el resultado es óptimo sobre ese grafo, no una garantía de ruta por carreteras reales.

4. Evidencia: Tijuana -> Cancún



Captura de la interfaz: la línea negra y sus nodos muestran el camino completo. El panel reporta costo, saltos y nodos expandidos.

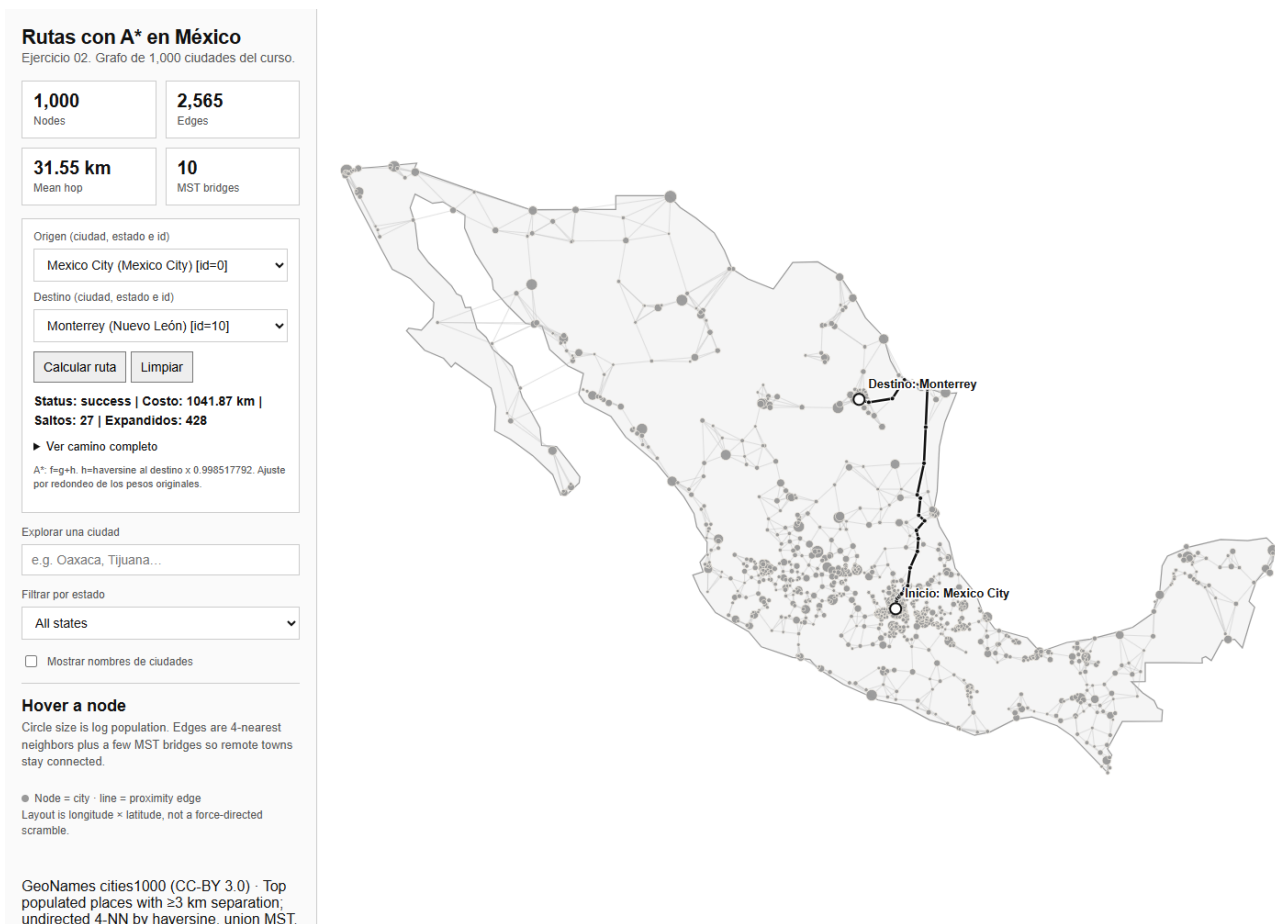
Ejecución de terminal

```
python find_route.py --from-city Tijuana --to Cancún
```

Resumen de la salida; el camino se abrevia aquí por espacio. La salida completa está en evidencias/tijuana-cancun-cli.txt.

```
Algorithm: A*
Heuristic: haversine x 0.998517792300
Status: success
Path: Tijuana[6] -> Villa del Prado 2da Sección[495] -> Terrazas del Valle[460] -> ... ->
      Tulum[494] -> Playa del Carmen[101] -> Cancún[25]
Depth: 124 hops
Cost: 4528.20 km
Expanded: 949
Generated: 4886
Frontier: 89 (max)
```

5. Evidencia: Mexico City -> Monterrey



Captura de la interfaz: la línea negra y sus nodos muestran el camino completo. El panel reporta costo, saltos y nodos expandidos.

Ejecución de terminal

```
python find_route.py --from-city "Mexico City" --to Monterrey
```

Resumen de la salida; el camino se abrevia aquí por espacio. La salida completa está en evidencias/mexico-city-monterrey-cli.txt.

```
Algorithm: A*
Heuristic: haversine x 0.998517792300
Status: success
Path: Mexico City[0] -> Gustavo Adolfo Madero[8] -> Puerto Escondido (Tepeolulco Puerto
      Escondido)[800] -> ... -> Cadereyta Jiménez[168] -> Jardines de la Silla (Jardines)[200] ->
      Monterrey[10]
Depth: 27 hops
Cost: 1041.87 km
Expanded: 428
Generated: 2174
Frontier: 56 (max)
```

6. Uso y verificación

Para ejecutar

Desde la carpeta Mexico map, se necesita Python 3.10 o posterior. No se instalan dependencias adicionales.

```
python find_route.py --from-city Tijuana --to Cancún
python find_route.py --from-city "Mexico City" --to Monterrey
python find_route.py --from-city 580 --to 25
python verificar.py
```

Para usar la interfaz, abrir mexico_map.html en Edge o Chrome. Elegir origen y destino, y pulsar Calcular ruta. El botón Limpiar borra el resultado. Ver camino completo despliega todas las ciudades. Los archivos HTML y route.js deben permanecer juntos.

Pruebas realizadas

Comprobación	Resultado
A* contra Dijkstra independiente	35 parejas con el mismo costo mínimo.
Costo de cada ruta	Se sumaron las aristas recorridas y se revisaron inicio, meta y profundidad.
CLI frente al navegador	Mismos caminos y costos en las dos parejas, otra pareja corta y las dos Puebla.
Casos especiales	Origen igual a destino: costo 0. Nombre inexistente: error. Nombre duplicado: aviso con opciones.
Mapa	Controles, limpiar, selección vacía, ruta completa y capturas de las dos parejas.
Datos originales	JSON idéntico al del profesor (comparación byte por byte). Sin regenerar 4-NN ni MST.

Procedencia

A* adaptado de [Búsqueda informada/project/search/astar.py](https://github.com/victorucetina/inteligencia-artificial). Mapa y datos del proyecto Mexico map del curso. Fuente de ciudades: GeoNames cities1000, licencia CC BY 3.0. Se conservan los créditos en el mapa y README.

<https://github.com/victorucetina/inteligencia-artificial> <https://www.geonames.org/> <https://creativecommons.org/licenses/by/3.0/>

Nota de entrega: esta carpeta contiene una copia del mapa para subir junto con el código. No ejecutar el generador sobre ella, porque reemplazaría la interfaz modificada.